4.1 Prototypowanie UI z Angular CLI i Angular Material: Od opisu tekstowego do gotowego komponentu Angular

Moduł 4: Szybkie Wdrażanie (Ship Fast) w oparciu o Angular

Krzysztof Rytlewski · 2026

Filozofia „Ship Fast" — dlaczego szybkie prototypowanie ma znaczenie?

Prototyp to nie produkt — jego celem jest jak najszybsze uwidocznienie idei, zebranie feedbacku i weryfikacja założeń. Dopiero po potwierdzeniu wartości warto inwestować w dopracowanie architektury.

Angular CLI i **Angular Material** tworzą zgrany tandem przyspieszający pracę:

- Angular CLI eliminuje konfigurację od zera — jedno polecenie generuje gotową strukturę projektu, komponenty, serwisy i routing.

- Angular Material dostarcza gotowe, dostępne (a11y) komponenty UI zgodne z Material Design, bez konieczności pisania CSS od podstaw.

Razem pozwalają przejść od pustego folderu do działającego interfejsu w ciągu minut — to jest esencja „Ship Fast".

Angular CLI — instalacja i tworzenie projektu

Angular CLI (`@angular/cli`) zarządza całym cyklem życia projektu Angular: tworzeniem, generowaniem kodu, uruchamianiem, testowaniem i budowaniem.

```
npm install -g @angular/cli
ng new my-ai-app --standalone --style=scss --routing=true
cd my-ai-app
ng serve
```

Flaga `--standalone` aktywuje architekturę bez NgModules (Angular 17+). Parametr `--dry-run` pozwala podejrzeć, jakie pliki zostaną wygenerowane, bez ich faktycznego tworzenia — niezastąpiony przy nauce.

Angular CLI — najważniejsze komendy generowania

Komenda	Skrót	Efekt
`ng generate component nazwa`	`ng g c nazwa`	Nowy komponent (ts, html, scss, spec)
`ng generate service nazwa`	`ng g s nazwa`	Nowy serwis
`ng generate pipe nazwa`	`ng g p nazwa`	Nowy pipe
`ng generate interface nazwa`	`ng g i nazwa`	Nowy interfejs TypeScript
`ng build --configuration production`	—	Build produkcyjny z optymalizacją

Przykład z `--dry-run`: `ng g c features/chat-window --dry-run` — wypisuje listę plików, które zostaną utworzone, nie tworząc ich.

Angular Material — biblioteka komponentów i instalacja

Angular Material to oficjalna biblioteka komponentów UI dla Angular, oparta na specyfikacji Material Design od Google. Zawiera ponad 35 gotowych komponentów: przyciski, formularze, karty, tabele, dialogi i nawigację.

```
ng add @angular/material
```

Polecenie `ng add` to więcej niż zwykły `npm install` — wykonuje **schemat** (*schematic*), który automatycznie importuje potrzebne moduły, dodaje style do `angular.json` i konfiguruje animacje. Kreator pyta o motyw kolorystyczny, typografię i animacje.

Angular Material — użycie w standalone component

W nowoczesnym Angular (v17+) każdy komponent standalone importuje potrzebne moduły Material bezpośrednio w dekoratorze `@Component`. Przykładowy komponent okna czatu korzysta z `MatCardModule`, `MatFormFieldModule`, `MatInputModule` i `MatButtonModule`.

Dwukierunkowy binding `[(ngModel)]` łączy pole textarea ze zmienną komponentu. Przycisk wywołuje metodę `send()` po kliknięciu. Cały komponent jest samowystarczalny — nie wymaga żadnego NgModule, by działać i wyświetlać formularz zgodny z Material Design.

Od opisu tekstowego do komponentu — workflow z AI

Kluczową umiejętnością jest skuteczne opisanie komponentu, by narzędzie AI (GitHub Copilot, Cursor, Claude) wygenerowało gotowy kod Angular Material nadający się do użycia od razu.

Dobry prompt dla AI zawiera:

- typ komponentu (formularz, lista, karta, dialog),
- bibliotekę (`Angular Material`),
- zachowanie (reaktywny formularz, walidacja, binding),
- pola i ich typy,
- opcjonalnie: styl układu (`flex`, `grid`, `mat-grid-list`).

Zasada review, don't retype — nie przepisujesz wygenerowanego kodu, weryfikujesz go i adaptujesz do potrzeb aplikacji.

Przykład skutecznego promptu dla AI

01

> *„Wygeneruj standalone Angular component z Angular Material: formularz rejestracyjny z polami Email (mat-input, walidacja email), Hasło (mat-input type=password, min 8 znaków), przyciskiem Submit (mat-raised-button color=primary). Użyj ReactiveFormsModule. Jeśli formularz jest niepoprawny, przycisk ma być wyłączony."*

Z takiego opisu AI generuje kompletny, działający komponent. Twoja rola to:

1. Ocena, czy komponent spełnia wymagania.
2. Drobna korekta (nazwy pól, stylizacja).
3. Integracja z resztą aplikacji.

Schematy (Schematics) i generowanie boilerplate'u z AI

Angular CLI opiera się na mechanizmie schematów — skryptów transformacji kodu tworzących i modyfikujących pliki projektu. Biblioteki (np. Angular Material) dostarczają własne schematy.

AI może wygenerować sekwencję komend CLI dla całego modułu:

```
ng g c features/chat/chat-shell --standalone
ng g c features/chat/message-list --standalone
ng g c features/chat/message-input --standalone
ng g s features/chat/chat
ng g i features/chat/message
```

Zamiast ręcznie tworzyć każdy plik, opisujesz funkcjonalność w języku naturalnym — AI tłumaczy to na sekwencję komend CLI, dramatycznie przyspieszając inicjalizację nowych modułów.

Lokalne środowisko deweloperskie z Dockerem

Podczas prototypowania warto uruchomić całe środowisko za pomocą Dockera — nowy deweloper startuje jedną komendą bez ręcznej konfiguracji.

Typowy `docker-compose.yml` dla środowiska deweloperskiego zawiera trzy serwisy:

- **frontend** — Angular z hot-reload; wolumen montuje kod źródłowy hosta, osobny wolumen anonimowy chroni `node_modules` kontenera przed nadpisaniem przez hosta.

- **backend** — Node.js/Express na porcie 3000; zmienna środowiskowa wskazuje na MongoDB w wewnętrznej sieci Docker.

- **mongo** — oficjalny obraz `mongo:7` na porcie 27017.

Uruchomienie całego stacku: `docker compose up --build`.

Oddzielenie środowisk: development vs produkcja

`Dockerfile.dev` jest przeznaczony **wyłącznie do developmentu** — zawiera `ng serve` z `--poll` (hot-reload) i montowaniem wolumenów, co nie ma sensu w produkcji.

W produkcji stosujemy inną architekturę:

- **Frontend** → statyczny bundle Angular wgrany na **AWS S3** i serwowany przez **CloudFront**.

- **Backend** → Node.js uruchamiany jako **AWS Lambda** (serverless).

Separacja środowisk chroni przed przypadkowym wdrożeniem konfiguracji deweloperskiej (np. otwarte porty, debug flags) na środowisko produkcyjne.

Najlepsze praktyki przy szybkim prototypowaniu

- **Standalone components** (Angular 17+) — brak NgModules, prostszy import, szybszy start projektu.
- **Lazy loading** od pierwszego dnia — `loadComponent` w routingu zapobiega kosztownej refaktoryzacji przed produkcją.
- **Angular CDK Layout** (`BreakpointObserver`) — responsywność bez konieczności pisania media queries w CSS.
- **`ng serve --open`** — automatyczne otwieranie przeglądarki przy starcie serwera deweloperskiego.
- **Hot Module Replacement** domyślnie włączony w Angular 17+ — zmiany w komponentach widoczne natychmiast bez pełnego przeładowania strony.
- Aliasy Bash (`alias ngs='ng serve --open'`) przyspieszają powtarzalne operacje w terminalu.